StayAI Hotel Booking — Complete Code Guide & Viva Prep

Oct 17, 2026 · @BizDecoded

1. Project overview

StayAI is a MERN-stack hotel booking app with three roles (user, owner, admin) and one AI feature: a guest types a sentence like "luxury hotel in Goa with pool", Google Gemini turns it into JSON filters, and the backend runs those filters as a MongoDB query.

What each role can do:

- **User (guest):** register, log in, browse all hotels, search with AI, book a hotel.
- **Owner:** everything a user can, plus create hotels (with an image), see their own listings, and see a dashboard with total hotels, rooms, bookings and revenue.
- **Admin:** has a dashboard page (currently static dummy data) and is the only role allowed on the delete-hotel route.

Tech stack

Layer	Technology	Why it is used
Frontend	React 18 + Vite 5	Component UI; Vite gives a fast dev server and build
Styling	Tailwind CSS 4 (via `@tailwindcss/vite`)	Utility classes written directly in JSX
Routing	react-router-dom 7	Client-side pages: `/login`, `/owner`, `/book/:id` …
Global state	Zustand 5	Stores the logged-in user; simpler than Redux
HTTP client	Axios	Calls the backend with `withCredentials: true` so cookies travel
Icons	lucide-react	MapPin, Star, BedDouble etc. on cards

Layer	Technology	Why it is used
Backend	Node.js + Express 5	REST API
Database	MongoDB + Mongoose 9	Schemas, validation, queries, `populate`
Auth	jsonwebtoken + bcryptjs + cookie-parser	JWT in an httpOnly cookie; passwords hashed
File upload	multer (memoryStorage)	Reads the hotel image into RAM as a Buffer
Image hosting	ImageKit (`@imagekit/nodejs`)	Stores the image, returns a public URL
AI	Google Gemini (`@google/generative-ai` , model `gemini-2.5-flash-lite`)	Natural-language search → JSON filters
Deployment	Netlify (frontend), Render (backend)	URLs hard-coded in `app.js` CORS and `src/api/*.js`

Folder structure

```
AI-powered-Hotel-Booking/
├── Backend/
|   ├── server.js                ← entry point: loads .env, connects DB,
starts server
|   └── src/
|       ├── app.js               ← Express app: CORS, JSON, cookies,
routers
|       ├── config/   db.js, gemini.js
|       ├── models/   user, hotel, book, blacklist
|       ├── middleware/ auth.middleware.js, authorize.js
|       ├── routes/   user.route, hotel.routes, owner.route
|       ├── controllers/ user, hotel, owner.dashboard, aiSearch
|       └── services/ imageKit.service.js, ai.services.js (empty)
└── frontend/
    ├── index.html, vite.config.js
    └── src/
        ├── main.jsx, App.jsx, AuthStore.js
        ├── api/       auth.js, hotel.js, owner.js
        ├── routes/    AppRoutes.jsx
```

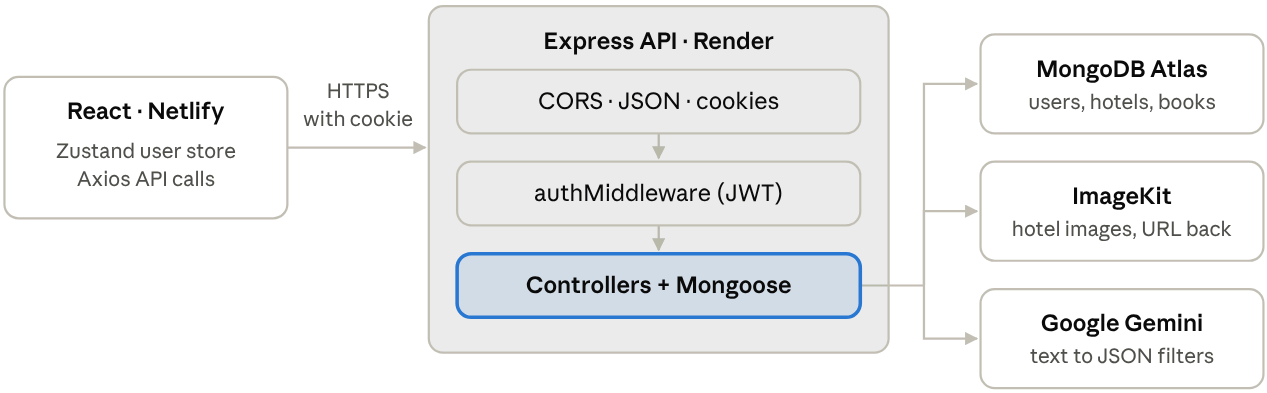
```
        ├── componenet/ Navbar, HotelCard, FormInput
        └── pages/     Login, Register, CreateHotel, BookHotel, HotelList,
Listing,
                       Dashboard/User, Dashboard/Owner, Dashboard/Admin
```

The backend follows the **MVC-style layered pattern**: route → middleware → controller → model. Services hold calls to outside systems (ImageKit), and config holds connections (MongoDB, Gemini). There is no View layer on the server because React is the view.

How one request travels (example: booking)

1. `BookHotel.jsx` calls `bookHotel(id, payload)` in `api/hotel.js`.
2. Axios sends `POST https://…onrender.com/api/hotel/book/<hotelId>`; the browser attaches the `token` cookie because of `withCredentials: true`.
3. Express runs global middleware in `app.js`: `cors` → `express.json()` → `cookieParser()`.
4. `app.use('/api/hotel', hotelRouter)` hands the request to `hotel.routes.js`, which matches `POST /book/:hotelId`.
5. `authMiddleware` reads `req.cookies.token`, verifies it with `JWT_SECRET`, and sets `req.userId` and `req.role`.
6. `bookingHotelController` validates input, loads the hotel, computes nights and price, creates a booking document, decrements `hotel.room`, and returns `201`.
7. The frontend shows "Booking Confirmed" and navigates to `/`.

Protected requests pass CORS and the JWT check before any controller runs



system architecture · frontend, API layers, 3 external services

The browser only talks to Express; Express alone talks to MongoDB, ImageKit and Gemini, so no secret key ever reaches the frontend.

2. Database schemas (Mongoose models)

The app has four collections: `users`, `hotels`, `books` and `tokenblacklists`. A booking links one user to one hotel; a hotel links back to the owner who created it.

How a model name becomes a collection name: `mongoose.model('hotel', schema)` → Mongoose lowercases and pluralises the name → collection `hotels`. So `'user'` → `users`, `'book'` → `books`, `'tokenBlackList'` → `tokenblacklists`.

Relationships (the ER picture):

- `user (owner) 1 ──< many hotel` via `hotel.createdBy`
- `user (guest) 1 ──< many book` via `book.userId`
- `hotel 1 ──< many book` via `book.hotelId`
- `tokenBlackList` stands alone (no references).

These are **references** (normalised), not embedded documents. A reference stores only the other document's `_id` (`type: mongoose.Schema.Types.ObjectId`), and `ref: "user"` tells `.populate()` which model to load it from. The string in `ref` must equal the name passed to `mongoose.model(...)`.

2.1 User schema — `models/user.model.js`

```
const userSchema = new mongoose.Schema({
  username: { type: String, required: [true, "username is required"] },
  email:    { type: String, required: [true, "email is required"], unique:
true },
  password: { type: String, required: [true, "password is required"] /*
select:false commented */ },
  role:     { type: String, enum: ["user","owner","admin"], default: "user" }
})
const userModel = mongoose.model('user', userSchema);
```

Field	Type	Rules	Meaning
`_id`	ObjectId	auto	Primary key added by MongoDB
`username`	String	required (custom message)	Display name

Field	Type	Rules	Meaning
`email`	String	required, `unique`	Login identity; `unique` builds a unique index so two users cannot share it
`password`	String	required	Stores the **bcrypt hash**, never plain text
`role`	String	`enum` user/owner/admin, default `user`	Drives redirects and authorisation
`__v`	Number	auto	Mongoose version key

Points to know:

- `required: [true, "msg"]` is the array form: the second element is the error message shown when validation fails.
- `unique: true` is **not a validator**; it creates a MongoDB unique index. A duplicate insert fails with error code `11000` (E11000 duplicate key). The controller also checks `findOne({email})` first to send a clean 409.
- `enum` rejects any value outside the list at save time.
- `select: false` is commented out. If enabled, `find()` would hide the password unless you write `.select('+password')`. Today the hash is returned by every user query (it is not sent to the client because the controller builds its own response object).
- No `timestamps` option here, so users have no `createdAt` / `updatedAt`.

2.2 Hotel schema — `models/hotel.model.js`

```
const hotelSchema = new mongoose.Schema({
  title:       { type: String, unique: true, required: true },
  description: { type: String, required: true },
  city:        { type: String, required: true, index: true },
  address:     { type: String },
  price:       { type: Number, required: true, index: true },
  rating:      { type: Number, default: 0, min: 0, max: 5 },
  totalReviews:{ type: Number, default: 0 },
  room:        { type: Number, required: true },
  maxGuests:   { type: Number, default: 2 },
  amenities:   [{ type: String, lowercase: true }],
```

```
  tags:        [{ type: String, lowercase: true }],
  nearbyPlaces:[{ type: String, lowercase: true }],
  hotelType:   { type: String, enum:
["budget","luxury","business","resort","hostel","villa"] },
  checkInTime: { type: String },
  checkOutTime:{ type: String },
  policies:    [{ type: String, lowercase: true }],
  img:         { type: String, required: true },
  createdBy:   { type: mongoose.Schema.Types.ObjectId, ref: "user" },
  isAvailable: { type: Boolean, default: true },
  featured:    { type: Boolean, default: false },
}, { timestamps: true });
const hotelModel = mongoose.model('hotel', hotelSchema);
```

Field	Type	Rules	Meaning / used by
`title`	String	required, unique	Hotel name; unique index blocks two hotels with the same name
`description`	String	required	Shown on the card (2-line clamp)
`city`	String	required, **indexed**	Saved lowercase by the controller; AI search filters on it
`address`	String	optional	Street address
`price`	Number	required, **indexed**	Price per night per room; AI `maxPrice` uses `$lte` on it
`rating`	Number	default 0, min 0, max 5	Star rating; AI `rating` uses `$gte`
`totalReviews`	Number	default 0	Placeholder; never updated in code
`room`	Number	required	Rooms currently available; **decremented on every booking**
`maxGuests`	Number	default 2	Displayed only; not enforced on booking
`amenities`	[String]	each lowercased	e.g. `["wifi","pool"]`; AI uses `$all`

Field	Type	Rules	Meaning / used by
`tags`	[String]	each lowercased	e.g. `["family","romantic"]`; AI uses `$in`
`nearbyPlaces`	[String]	each lowercased	e.g. `["beach","airport"]`; AI uses `$in`
`hotelType`	String	enum of 6 values	Same 6 values as the `<select>` in CreateHotel
`checkInTime` / `checkOutTime`	String	optional	From `<input type="time">`, e.g. `"14:00"`
`policies`	[String]	each lowercased	e.g. `["no smoking"]`
`img`	String	required	ImageKit URL returned after upload
`createdBy`	ObjectId → user	optional	Owner's id; used by listing + owner dashboard
`isAvailable`	Boolean	default true	Not used in any query yet
`featured`	Boolean	default false	Shows a black "Featured" badge on HotelCard
`createdAt` / `updatedAt`	Date	from `timestamps: true`	Used to sort newest first

Points to know:

- `[{ type: String, lowercase: true }]` declares an **array of strings**; the `lowercase` setter runs on every element before saving.
- `index: true` on `city` and `price` creates **single-field B-tree indexes** so filters on those fields avoid a full collection scan.
- `{ timestamps: true }` (second argument to `Schema`) adds and maintains `createdAt` and `updatedAt` automatically.
- The large commented block at the top is an unused **Room schema** idea (per-room type, price and `unavailableDates`). It shows the planned path to real date-wise availability.

2.3 Booking schema — `models/book.model.js`

```
const bookingSchema = new mongoose.Schema({
  userId:  { type: mongoose.Schema.Types.ObjectId, ref: "user",  required:
true },
  hotelId: { type: mongoose.Schema.Types.ObjectId, ref: "hotel", required:
true },
  checkIn:  { type: Date, required: true },
  checkOut: { type: Date, required: true },
  totalPrice:    { type: Number, required: true },
  selectedRooms: { type: Number, required: true, min: 1 },
  guests:        { type: Number, default: 1 },
  bookingStatus: { type: String, enum:
["pending","confirmed","cancelled","completed"], default: "confirmed" },
  specialRequests: { type: String, default: "" },
  pricePerNight: { type: Number },
  totalNights:   { type: Number },
}, { timestamps: true });
bookingSchema.index({ userId: 1 });
bookingSchema.index({ hotelId: 1 });
const bookingModel = mongoose.model("book", bookingSchema);
```

Field	Type	Rules	Meaning
`userId`	ObjectId → user	required, indexed	Who booked
`hotelId`	ObjectId → hotel	required, indexed	Which hotel
`checkIn` / `checkOut`	Date	required	Stay dates
`totalPrice`	Number	required	`price × nights × rooms`, computed on the server
`selectedRooms`	Number	required, `min: 1`	Rooms booked
`guests`	Number	default 1	Number of guests
`bookingStatus`	String	enum of 4, default `confirmed`	Lifecycle state; only `confirmed` is ever set
`specialRequests`	String	default `""`	Free text, e.g. early check-in

Field	Type	Rules	Meaning
`pricePerNight`	Number	optional	**Price snapshot** at booking time
`totalNights`	Number	optional	Nights between the two dates

Points to know:

- **Price snapshot:** `pricePerNight` copies the hotel price at booking time, so if the owner changes the price later, old bookings keep the price the guest actually paid.
- `bookingSchema.index({ userId: 1 })` and `({ hotelId: 1 })` are **schema-level indexes** (`1` = ascending). The `hotelId` index speeds up the owner dashboard's `find({ hotelId: { $in: hotelIds } })`.
- `ref` values (`"user"`, `"hotel"`) let the owner dashboard call `.populate("userId", "username email")`.

2.4 Token blacklist schema — `models/blacklist.model.js`

```
const tokenBlackListSchema = new mongoose.Schema({
  token: { type: String, required: [true, "token is required to blackList"],
           unique: [true, "token already blacklisted"] }
}, { timestamps: true })
tokenBlackListSchema.index({ createdAt: 1 }, { expireAfterSeconds: 60*60*24*3
}) // 3 days
const tokenBlackListModel = mongoose.model("tokenBlackList",
tokenBlackListSchema);
module.exports = { tokenBlackListModel }
```

Field	Type	Rules	Meaning
`token`	String	required, unique	A JWT that was logged out
`createdAt`	Date	from timestamps, **TTL index**	MongoDB deletes the document 3 days after this

Points to know:

- **Why a blacklist?** A JWT is stateless: once signed it stays valid until it expires. Logging out only deletes the cookie in that browser; a copied token would still work. Saving it in a blacklist lets the server refuse it.

- **TTL (time-to-live) index:** `expireAfterSeconds: 259200` tells MongoDB's background job (runs about every 60 s) to delete each document 3 days after `createdAt`. 3 days matches the JWT `expiresIn: "3d"`, so a blacklisted token is removed exactly when it would have expired anyway.
- `unique: [true, "msg"]` — the custom message is ignored because `unique` is an index, not a validator.
- **Gap:** `authMiddleware` never checks this collection, so the blacklist is written but never read (see section 7).
- It is exported as an object `{ tokenBlackListModel }`, so it is imported with destructuring, unlike the other models.

3. Backend line by line — setup, config, middleware, routes, services

These files wire the server together; the business logic lives in the controllers (section 4).

3.1 `Backend/package.json`

- `"main": "server.js"` — the entry file.
- `"type": "commonjs"` — the backend uses `require` / `module.exports`, not `import` / `export`.
- `scripts` has only a placeholder `test`; there is no `start` or `dev` script, so it is started with `node server.js`.
- Dependencies: `express` (server), `mongoose` (MongoDB ODM), `bcryptjs` (hashing), `jsonwebtoken` (JWT), `cookie-parser` (reads cookies), `cors` (cross-origin), `dotenv` (reads `.env`), `multer` (multipart uploads), `@imagekit/nodejs` (image hosting), `@google/generative-ai` (Gemini). `path` is listed but is a built-in Node module, so that install is unnecessary.

3.2 `server.js` — entry point

```
1  require("dotenv").config();
3  const app = require("./src/app");
4  const connectDB = require("./src/config/db");
7  connectDB();
9  const PORT = 3000;
11 app.listen(PORT, () => {
12     console.log(`Server running on port ${PORT}`);
13 });
```

- **L1** loads variables from `.env` into `process.env` (`MONGO_URI`, `JWT_SECRET`, `IMAGEKIT_PRIVATE_KEY`, `GOOGLE_GENAI_API_KEY`). It must run first so every later file can read them.
- **L3** imports the configured Express app (separating `app` from `server` makes the app testable without opening a port).
- **L4** imports the DB connection function.
- **L7** starts connecting to MongoDB. It is not awaited, so the server may start accepting requests a moment before the DB is ready (Mongoose buffers queries until connected, so this usually works).
- **L9** hard-codes port 3000. On Render you would normally use `process.env.PORT || 3000`.
- **L11–13** starts the HTTP server and logs the port.

3.3 `src/app.js` — Express app

```
1  const express=require('express')
2  const app=express();
3  const cookieParser=require('cookie-parser')
4  const cors=require('cors')
6  app.use(cors({
7      origin:[ 'http://localhost:5173', 'https://singular-cannoli-
2a3fd2.netlify.app' ],
11     credentials:true
12 }))
15 const AuthRouter=require("../src/routes/user.route")
16 const hotelRouter=require("../src/routes/hotel.routes")
17 const ownerRouter=require("./routes/owner.route")
20 app.use(express.json())
21 app.use(cookieParser());
24 app.use('/api/auth',AuthRouter);
26 app.use('/api/hotel',hotelRouter);
28 app.use('/api',ownerRouter)
38 module.exports=app;
```

- **L1–2** import Express and create the app object.
- **L3–4** import the cookie parser and CORS middleware.
- **L6–12 CORS:** the frontend (Netlify or local Vite on port 5173) is a different origin from the backend (Render), so the browser blocks responses unless the server sends `Access-Control-Allow-Origin`. `origin` is a whitelist of the two allowed sites. `credentials: true` sends `Access-Control-Allow-Credentials: true`, which is required for the browser to send and accept cookies cross-origin. With credentials, the origin cannot be `*`; it must be listed exactly.
- **L15–17** import the three routers. `"../src/routes/..."` and `"./routes/..."` resolve to the same folder; the first is just a roundabout path.
- **L20** **`express.json()`** parses a JSON request body into `req.body`. Without it `req.body` is `undefined`. (Multipart form data for hotel creation is parsed later by multer, not by this.)
- **L21** **`cookieParser()`** parses the `Cookie` header into `req.cookies`, so `req.cookies.token` works.
- **L24–28 mounting:** every route in `user.route.js` is prefixed with `/api/auth`, in `hotel.routes.js` with `/api/hotel`, and in `owner.route.js` with `/api` (so `/owner` becomes `/api/owner`).

- **Order matters:** middleware runs top to bottom, so CORS, JSON and cookie parsing happen before any router.
- **L38** exports the app for `server.js`.

3.4 `src/config/db.js` — MongoDB connection

```
1  require('dotenv').config()
3  const mongoose=require('mongoose')
5  console.log(process.env.MONGO_URI);
7  async function connectDB(){
8      try{
10         await mongoose.connect(process.env.MONGO_URI);
11         console.log("Database is connected")
12     }catch(err){
13         console.log(err);
14     }
17 }
19 module.exports=connectDB;
```

- **L1** loads `.env` again (harmless; already loaded by `server.js`).
- **L5** prints the full connection string, including the DB password, to the logs — a security leak that should be removed.
- **L7–17** `connectDB` is `async` because `mongoose.connect` returns a Promise. `await` waits for the connection; on failure the `catch` only logs, so the server keeps running without a database. A stricter version calls `process.exit(1)`.
- **L19** exports the function.

3.5 `src/config/gemini.js` — AI model setup

```
1  const path=require('path');
3  require("dotenv").config({ path: path.resolve(__dirname, "../.env") });
7  const express = require("express");
8  const { GoogleGenerativeAI } = require("@google/generative-ai");
12 console.log(process.env.GOOGLE_GENAI_API_KEY);
14 const genAI = new GoogleGenerativeAI(process.env.GOOGLE_GENAI_API_KEY);
19 const model = genAI.getGenerativeModel({
22   model: "gemini-2.5-flash-lite",
23   generationConfig: { responseMimeType: "application/json" },
27 });
29 module.exports = model;
```

- **L1–3** loads `.env` from `src/.env` (`__dirname` is `src/config`, so `../.env` = `src/.env`). The real `.env` is in `Backend/`, but it was already loaded by `server.js`, so this line does nothing harmful.
- **L7** imports Express but never uses it — dead code.
- **L8** imports the SDK class with destructuring.
- **L12** prints the API key to the logs — remove it.
- **L14** creates a Gemini client authenticated with the key.
- **L19–27** creates a model handle. `gemini-2.5-flash-lite` is a small, fast, cheap model, a good fit for simple parsing. `responseMimeType: "application/json"` is **JSON mode**: Gemini is told to reply with JSON only, which makes `JSON.parse` far more reliable.
- **L29** exports one shared model object; Node caches modules, so it is created once and reused for every request (singleton).

3.6 `src/middleware/auth.middleware.js` — authentication

```
1  require("dotenv").config();
3  const jwt = require("jsonwebtoken");
5  async function authMiddleware(req, res, next) {
7      try {
9          const token = req.cookies?.token;
11         if (!token) {
12             return res.status(401).json({ success: false, message: "Token
missing" });
16         }
18         const decoded = jwt.verify(token, process.env.JWT_SECRET);
23         req.userId = decoded.userId;
24         req.role = decoded.role;
26         next();
28     } catch (err) {
30         console.log("Auth middleware error:", err.message);
32         return res.status(401).json({ success: false, message: "Invalid or
expired token" });
36     }
37 }
39 module.exports = authMiddleware;
```

- A **middleware** is a function `(req, res, next)` that runs between the request and the controller. It either ends the request (sends a response) or calls `next()` to pass control on.

- **L9** reads the JWT from the `token` cookie. `?.` (optional chaining) avoids a crash if `req.cookies` is undefined.
- **L11–16** no token → `401 Unauthorized`. `return` stops the function so `next()` never runs.
- **L18** `jwt.verify` checks the signature with the secret and checks `exp`. If the token was altered, signed with another key, or expired, it **throws** — caught below.
- **L23–24** puts the payload values on `req` so later handlers know who is calling. These were put into the token at login: `{ userId, role }`.
- **L26** `next()` passes control to the next middleware or controller.
- **L28–36** any verify error → `401 Invalid or expired token`.
- Not checked: the token blacklist (logout does not truly revoke tokens).

3.7 `src/middleware/authorize.js` — role-based access

```
3  const authorize = (...roles) => {
5      return (req, res, next) => {
7      if (!roles.includes(req.user.role)) {
9          return res.status(403).json({ message: "Access denied" })
12     }
14     next()
15     }
16 }
18 module.exports=authorize
```

- This is a **middleware factory** (a higher-order function / closure): `authorize("admin")` returns a new middleware that remembers `roles = ["admin"]`.
- **L3** `...roles` is a rest parameter, so you can write `authorize("admin", "owner")`.
- **L7** checks if the caller's role is in the allowed list; if not, **403 Forbidden** (logged in but not allowed — different from 401, not logged in).
- **Bug:** it reads `req.user.role`, but `authMiddleware` sets `req.role`, and `req.user` is never set. `req.user` is `undefined`, so `req.user.role` throws a TypeError and the delete route always fails with a 500. Fix: `roles.includes(req.role)`.

3.8 Routes

`routes/user.route.js` (mounted at `/api/auth`)

```
1  const express=require('express')
2  const router=express.Router();
3  const
{registerController,loginController,getMeController,logoutController}=require
('../controllers/user.controller')
4  const authMiddleware=require('../middleware/auth.middleware')
7  router.post('/register',registerController)
9  router.post('/login',loginController)
11 router.get('/me',authMiddleware,getMeController)
13 router.post('/logout',authMiddleware,logoutController)
14 module.exports=router
```

- `express.Router()` creates a mini-app that holds a group of routes; `app.use(prefix, router)` mounts it.
- Register and login are public. `/me` and `/logout` run `authMiddleware` first. Handlers listed after the path run in order.

`routes/hotel.routes.js` (mounted at `/api/hotel`)

```
3  const
{createHotelController,getHotelController}=require('../controllers/hotel.cont
roller')
4  const
{bookingHotelController,deleteHotelController,listingController}=require("../
controllers/hotel.controller")
5  const {aiSearchController}=require("../controllers/aiSearch.controller")
6  const authMiddleware=require("../middleware/auth.middleware")
7  const {uploadFile}=require('../services/imageKit.service')
8  const multer=require('multer')
9  const authorize=require('../middleware/authorize')
10 const upload=multer({ storage:multer.memoryStorage() })
14
router.post('/create',authMiddleware,upload.single('img'),createHotelControll
er)
15 router.get('/getHotel',getHotelController)
18 router.post('/book/:hotelId',authMiddleware,bookingHotelController)
21
router.delete('/book/:hotelId',authMiddleware,authorize("admin"),deleteHotelC
ontroller)
24 router.get('/listing',authMiddleware,listingController)
28 router.post("/ai-search", aiSearchController);
37 module.exports=router
```

- **L3–4** import controllers in two lines from the same file (could be one).
- **L7** imports `uploadFile` but the route file never uses it (the controller imports it itself).
- **L10** `multer.memoryStorage()` keeps the uploaded file in RAM as `req.file.buffer` instead of writing it to disk. Good for forwarding straight to ImageKit; no `limits.fileSize` or `fileFilter` is set, so any file of any size is accepted.
- **L14** `upload.single('img')` parses a `multipart/form-data` body: the file field named `img` goes to `req.file`, all text fields go to `req.body` (as strings). Chain: login check → parse upload → create. No `authorize("owner")`, so any logged-in user can create a hotel.
- **L15** public list of all hotels.
- **L18** `:hotelId` is a **route parameter**, read as `req.params.hotelId`.
- **L21** same path as L18 but `DELETE` method, so Express treats it as a different route. It deletes the **hotel**, not a booking, so the path name is misleading.
- **L24** owner's own listings.
- **L28** AI search is public (no `authMiddleware`), so anyone can spend the Gemini quota; a rate limiter would help.

`routes/owner.route.js` (mounted at `/api`)

```
4  const
{ownerDashboard}=require("../controllers/owner.dashboard.controller")
5  const authMiddleware=require("../middleware/auth.middleware")
7  router.get("/owner", authMiddleware, ownerDashboard);
13 module.exports = router;
```

- Full path `GET /api/owner`. Login required but no owner role check; a normal user just gets zeros because they own no hotels.

3.9 Services

`services/imageKit.service.js`

```
1  require('dotenv').config()
2  const {ImageKit}=require("@imagekit/nodejs");
5  const ImageKitClient=new ImageKit({
privateKey:process.env.IMAGEKIT_PRIVATE_KEY });
9  async function uploadFile(file){
11     const result=await ImageKitClient.files.upload({
12         file,
13         fileName:"image_"+Date.now(),
14         folder:"Hotel/Image"
15     })
16     return result;
17 }
19 module.exports= {uploadFile};
```

- **L5** creates one ImageKit client authenticated with the private key (server-side only; never send it to the browser).
- **L9** `file` is the image as a **base64 string** (the controller converts the Buffer).
- **L13** unique-ish name using the current timestamp in milliseconds.
- **L14** stores images in the `Hotel/Image` folder in ImageKit.
- **L16** returns the upload result; the controller saves `result.url` into `hotel.img`.
- **Why a separate service?** Separation of concerns: if you switch to Cloudinary or S3, only this file changes.

`services/ai.services.js` is empty. The Gemini logic lives in `config/gemini.js` + `aiSearch.controller.js`; ideally the prompt-building and parsing would move into this service.

4. Backend controllers line by line

Controllers hold the logic; every one is an `async (req, res)` function wrapped in `try/catch` so a thrown error becomes a JSON 500 instead of crashing the server.

4.1 `user.controller.js` — register, login, me, logout

Imports (L1–4): `userModel`, `bcrypt` (hashing), `jwt` (tokens), `{ tokenBlackListModel }` (logout).

registerController (L6–99)

```
10  const { username, email, password, role } = req.body;
18  if (!username || !email || !password) { return
res.status(400).json({..."Username, email or password missing"}) }
27  const isUserExist = await userModel.findOne({ email });
31  if (isUserExist) { return res.status(409).json({..."User already
exists"}) }
40  const allowedRoles = ["user", "owner"];
42  const userRole = allowedRoles.includes(role) ? role : "user";
47  const hash = await bcrypt.hash(password, 10);
50  const user = await userModel.create({ username, email, password: hash,
role: userRole });
58  const token = jwt.sign({ userId: user._id, role: user.role },
process.env.JWT_SECRET, { expiresIn: "3d" });
70  res.cookie("token", token, { httpOnly: true, secure: true, sameSite:
"none", maxAge: 3*24*60*60*1000 });
78  return res.status(201).json({ success: true, message: "User registered
successfully",
82      user: { id: user._id, username: user.username, email: user.email,
role: user.role } });
90  } catch (err) { console.log(err); return res.status(500).json({..."Server
error in register controller"}) }
```

- **L10** destructures the JSON body (parsed by `express.json()`).
- **L18** presence check; `400 Bad Request` if a field is missing.
- **L27–37** looks up the email; if found, `409 Conflict` (resource already exists).
- **L40–44 privilege-escalation guard:** the client chooses `user` or `owner` from a dropdown, but anyone could send `role: "admin"` with Postman. The whitelist makes any other value fall back to `user`, so admins can only be created directly in the database.
- **L47** `bcrypt.hash(password, 10)` — 10 is the **salt rounds / cost factor** (2^10 = 1024 iterations). bcrypt generates a random **salt** and stores it inside the hash string (`$2a$10$<22-char salt><31-char hash>`), so two users with the same password get different hashes, which defeats rainbow tables. It is deliberately slow to make brute force expensive.
- **L50** inserts the user with the hash, never the plain password.

- **L58** `jwt.sign(payload, secret, options)` builds a token: `header.payload.signature`, each part base64url-encoded. The payload `{ userId, role }` is readable by anyone (not encrypted), only **signed** (HMAC-SHA256 by default) so it cannot be changed without the secret. `expiresIn: "3d"` adds an `exp` claim.
- **L70 cookie options:**
  - `httpOnly: true` — JavaScript (`document.cookie`) cannot read it, which protects the token from XSS theft.
  - `secure: true` — sent only over HTTPS.
  - `sameSite: "none"` — needed because Netlify and Render are different sites; `"none"` requires `secure: true`.
  - `maxAge` — 3 days in milliseconds, matching the JWT expiry.
- **L78–88** `201 Created` with a safe user object (no password). The user is logged in immediately after registering.

loginController (L101–158)

```
103 const { email, password } = req.body;
105 if (!email || !password) return res.status(400).json({ message: "Email
and password required" });
111 const user = await userModel.findOne({ email });
113 if (!user) return res.status(404).json({ message: "User does not exist"
});
119 const pass = await bcrypt.compare(password, user.password);
121 if (!pass) return res.status(401).json({ message: "Password does not
match" });
127 const token = jwt.sign({ userId: user._id, role: user.role },
process.env.JWT_SECRET, { expiresIn: "3d" });
136 res.cookie("token", token, { httpOnly: true, secure: true, sameSite:
"none" });
144 return res.status(200).json({ user: { _id: user._id, name: user.username,
email: user.email, role: user.role } });
153 } catch (err) { return res.status(500).json({ message: err.message }); }
```

- **L119** `bcrypt.compare` reads the salt out of the stored hash, hashes the typed password with it, and compares. You never "decrypt" a hash — hashing is one-way.
- **L113 / L121** separate messages ("does not exist" vs "does not match") reveal which emails are registered (**user enumeration**). Best practice is one generic "Invalid email or password" with 401.

- **L136** no `maxAge`, so this is a **session cookie** that disappears when the browser closes, though the JWT inside is valid for 3 days. Register sets `maxAge`; login does not — inconsistent.
- **L144** returns the user including `role`; the frontend uses `role` to choose the dashboard. Note the key is `name` here but `username` in register.

getMeController (L159–174)

```
161 const user=await userModel.findById(req.userId)
163 if(!user){ res.json({ message:"No user exist try to register yourself" })
}
169 res.status(200).json({ message:"User fetched succesfully",
username:user.username, email:user.email })
```

- Purpose: "who am I?" — used on page refresh to restore the login from the cookie.
- **L161** `req.userId` came from `authMiddleware`.
- **Bug 1:** L163 has no `return`; if the user was deleted, it sends a response and then L169 reads `user.username` on `null`, which throws (and there is no try/catch here).
- **Bug 2:** it returns `username` and `email` at the top level, but the frontend's `fetchUser` reads `res.data.user`, which is `undefined`. It also does not return `role`. Result: after a refresh the store's user is always `null` and private pages redirect to login. Fix: `res.status(200).json({ user: { _id, username, email, role } })`.

logoutController (L176–196)

```
177 const token=req.cookies.token || req.headers.authorization?.split(" ")
[1];
179 if(!token){ return res.status(200).json({ message:"User logout
successfully" }) }
186 await tokenBlackListModel.create({ token:token });
190 res.clearCookie("token");
192 res.status(200).json({ message:"User logout successfully" })
```

- **L177** reads the token from the cookie, or from an `Authorization: Bearer <token>` header (splitting on the space and taking index 1).
- **L186** stores the token in the blacklist (removed by TTL after 3 days).
- **L190** `clearCookie("token")` tells the browser to delete the cookie. For cross-site cookies it should get the same options used when setting it (`secure`, `sameSite: "none"`), otherwise some browsers keep it.

- No try/catch: logging out twice with the same token violates the `unique` index and throws E11000.

4.2 `hotel.controller.js`

Imports (L1–3): `hotelModel`, `bookingModel`, `{ uploadFile }`.

createHotelController (L6–165)

```
12  const { title, description, price, city, room, address, rating,
maxGuests, hotelType,
25          amenities, tags, nearbyPlaces, policies, checkInTime,
checkOutTime, featured } = req.body;
38  if (!title || !description || !price || !city || !room) return
res.status(400).json({ message: "Required fields missing" });
51  if (!req.file) return res.status(400).json({ message: "Hotel image
required" });
61  let result;
63  try { result = await uploadFile(req.file.buffer.toString("base64")); }
71  catch (uploadErr) { return res.status(500).json({ message: "Image upload
failed" }); }
84  const amenitiesArray = amenities ? amenities.split(",").map(item =>
item.trim().toLowerCase()) : [];
90  const tagsArray = ...same...;  96 nearbyPlacesArray = ...;  102
policiesArray = ...;
109 const hotel = await hotelModel.create({
111   title, description, city: city.toLowerCase(), address,
119   price: Number(price), rating: Number(rating) || 0, room: Number(room),
125   maxGuests: Number(maxGuests) || 2, hotelType,
129   amenities: amenitiesArray, tags: tagsArray, nearbyPlaces:
nearbyPlacesArray, policies: policiesArray,
137   checkInTime, checkOutTime, featured: featured || false,
143   createdBy: req.userId, img: result.url,
147 });
149 return res.status(201).json({ message: "Hotel created successfully",
hotel });
157 } catch (err) { return res.status(500).json({ message: err.message ||
"Server Error" }); }
```

- **L12–35** the body comes from `FormData` (multipart), so **every value is a string** — `price` is `"2500"`, `featured` is `"true"` / `"false"`. That is why L119–125 wrap numbers in `Number(...)`.

- **L38–48** required-field check (`400`).
- **L51–55** multer put the file on `req.file` ; missing → 400.
- **L57–58** debug logs of body and file.
- **L61–78** nested try/catch so an ImageKit failure gives a specific message. `req.file.buffer` is raw bytes; `.toString("base64")` turns them into a text string ImageKit accepts.
- **L84–106** turns `"WiFi, Pool ,Breakfast"` into `["wifi","pool","breakfast"]` : `split(",")` → array, `trim()` removes spaces, `toLowerCase()` normalises case. Ternary returns `[]` if the field is empty. Lowercasing matters because MongoDB string matching is case-sensitive.
- **L115** city saved lowercase for consistent searching.
- **L121** `Number(rating) || 0` — `Number("")` is `0` and `Number(undefined)` is `NaN` ; both are falsy, so the default 0 is used.
- **L141** `featured: "false"` (a string) is truthy, but Mongoose's Boolean caster converts the string `"false"` to `false` , so it saves correctly.
- **L143** `createdBy: req.userId` — ownership comes from the verified token, not from the client body, so a user cannot create a hotel in someone else's name.
- **L145** stores only the image URL; images live on ImageKit's CDN, not in MongoDB.
- **L149** `201 Created` . A duplicate `title` throws E11000 and returns 500 with Mongo's message.

bookingHotelController (L166–309)

```
170 const { checkIn, checkOut, selectedRooms, guests, specialRequests } =
req.body;
178 const { hotelId } = req.params;
181 if (!checkIn || !checkOut || !selectedRooms) return 400 "Required fields
missing";
193 const hotel = await hotelModel.findById(hotelId);
195 if (!hotel) return 404 "Hotel not found";
203 const checkInDate = new Date(checkIn);
205 const checkOutDate = new Date(checkOut);
207 if (checkOutDate <= checkInDate) return 400 "Check-out must be after
check-in";
217 const totalNights = Math.ceil((checkOutDate - checkInDate) / (1000 * 60 *
60 * 24));
225 if (Number(selectedRooms) > hotel.room) return 400 "Requested rooms not
available";
236 const totalPrice = hotel.price * totalNights * Number(selectedRooms);
242 const bookingHotel = await bookingModel.create({ userId: req.userId,
hotelId,
249   checkIn: checkInDate, checkOut: checkOutDate, selectedRooms:
Number(selectedRooms),
256   guests: Number(guests) || 1, specialRequests: specialRequests || "",
262   totalPrice, totalNights, pricePerNight: hotel.price, bookingStatus:
"confirmed" });
277 hotel.room = hotel.room - Number(selectedRooms);
281 await hotel.save();
284 res.status(201).json({ success: true, message: "Hotel booked
successfully", booking: bookingHotel });
```

- **L178** the hotel id comes from the URL (`/book/:hotelId`).
- **L193** `findById` — if `hotelId` is not a valid 24-hex ObjectId, Mongoose throws a **CastError**, caught as 500 (ideally a 400 after `mongoose.isValidObjectId`).
- **L203–214** converts `"2026-10-20"` strings to Date objects. Comparing Dates with `<=` compares their millisecond timestamps.
- **L217** subtracting two Dates gives milliseconds; dividing by 86,400,000 (ms in a day) gives days; `Math.ceil` rounds up a partial day to a full night.
- **L225** compares against the hotel's current `room` count.
- **L236 price is computed on the server**, never trusted from the client (the frontend shows its own estimate with a hard-coded ₹2000, but the server ignores it).

- **L242–272** creates the booking with a price snapshot.
- **L277–281** reduces the hotel's available rooms. Comment says "temporarily", but nothing ever adds the rooms back, and it ignores dates: a room booked for one night in October is gone forever.
- **Race condition:** two requests can both read `room = 1`, both pass L225, and both book. An atomic update fixes it: `findOneAndUpdate({ _id: hotelId, room: { $gte: n } }, { $inc: { room: -n } })`. Creating the booking and decrementing should also be in one **transaction** so one cannot succeed without the other.

getHotelController (L311–345)

```
315 const hotels = await hotelModel.find().sort({ createdAt: -1 });
319 res.status(200).json({ success: true, message: "...", totalHotels:
hotels.length, hotels });
```

- `find()` with no filter returns every hotel; `sort({ createdAt: -1 })` is newest first (`-1` = descending). No pagination — fine for a demo, slow with thousands of hotels (use `.skip().limit()`).

deleteHotelController (L347–380)

```
351 const { hotelId } = req.params;
354 const hotel = await hotelModel.findById(hotelId);
356 if (!hotel) return 404 "Hotel not found";
364 await hotelModel.findByIdAndDelete(hotelId);
366 return res.status(200).json({ success: true, message: "Hotel deleted
successfully" });
```

- Checks existence first so it can return a clear 404, then deletes. Two queries; `findByIdAndDelete` alone returns `null` when not found, which would do it in one.
- Bookings for the deleted hotel stay behind as **orphans** (no cascade delete in MongoDB).
- Unreachable in practice because `authorize` crashes (section 3.7).

listingController (L382–411)

```
384 const token = req.cookies.token;
386 if (!token) return 401 "Token is missing";
393 const listing = await hotelModel.find({ createdBy: req.userId });
397 return res.status(200).json({ success: true, message: "...", hotels:
listing });
```

- **L384–391** redundant: `authMiddleware` already checked the token.
- **L393** returns only hotels whose `createdBy` equals the caller's id — "my listings".

Exports (L413): all five controllers in one object.

4.3 `owner.dashboard.controller.js`

```
4  const ownerDashboard = async (req, res) => {
8    const ownerId = req.userId;
11   const hotels = await hotelModel.find({ createdBy: ownerId });
15   const hotelIds = hotels.map(hotel => hotel._id);
18   const bookings = await bookingModel.find({ hotelId: { $in: hotelIds } })
22       .populate("userId", "username email")
23       .populate("hotelId", "title city");
26   const totalHotels = hotels.length;
28   const totalRooms = hotels.reduce((acc, hotel) => acc + hotel.room, 0);
32   const totalBookings = bookings.length;
34   const totalRevenue = bookings.reduce((acc, booking) => acc +
booking.totalPrice, 0);
38   res.status(200).json({ success: true, stats: { totalHotels, totalRooms,
totalBookings, totalRevenue }, hotels, bookings });
```

- **L11** all hotels owned by this owner.
- **L15** `map` builds an array of just their `_id`s.
- **L18** `$in` matches bookings whose `hotelId` is any id in that array — one query instead of a loop (avoids the N+1 problem).
- **L22–23** **`populate`** replaces the stored ObjectId with the referenced document, selecting only the listed fields. So `booking.userId` becomes `{ _id, username, email }` and `booking.hotelId` becomes `{ _id, title, city }`. Internally Mongoose runs one extra query per populated path. The frontend reads `booking.userId?.username`.

- **L28** `reduce` sums a field across an array; `0` is the starting value. `totalRooms` is **remaining** rooms, since bookings decrement it.
- **L34** revenue counts every booking regardless of status (cancelled bookings would still count if they existed).
- The same could be done in one **aggregation pipeline** (`$match` → `$lookup` → `$group`), which is faster at scale.

4.4 `aiSearch.controller.js` — the AI feature

Gemini does not search the database; it only **translates** English into a filter object. MongoDB does the actual search.

```
1   require("dotenv").config();
3   const model = require("../config/gemini");
6   const hotelModel = require("../models/hotel.model.js");
11  function escapeRegex(text) {
13    return text.replace(/[.*+?^${}()|[\]\\]/g, "\\$&");
17  }
```

- **L11–17** **`escapeRegex`:** puts a backslash before every regex special character (`. * + ? ^ $ { } ( ) | [ ] \`). `$&` in the replacement means "the matched character". Without it, a city like `"goa.*"` would act as a regex pattern; malicious patterns can also cause slow **ReDoS** (regular-expression denial of service).

```
28  const { prompt } = req.body;
31  if (!prompt) return res.status(400).json({ success: false, message:
"Search prompt required" });
44  if (prompt.length > 500) return res.status(400).json({ success: false,
message: "Prompt too long" });
```

- Validation; the 500-character cap limits cost and abuse.

```
57  const aiPrompt = `
59  You are a hotel search query parser.
61  Your ONLY task is to extract structured hotel filters from user input.
63  Return ONLY valid raw JSON.  Never explain anything. Never add markdown.
Never add extra text.
69  Allowed JSON fields only: city, hotelType, amenities, nearbyPlaces,
maxPrice, tags, rating
79  Rules: amenities/nearbyPlaces/tags must be array; maxPrice/rating must be
number
87  If field not found, omit it.
89  Example: { "city":"goa", "hotelType":"luxury", "amenities":
["pool","wifi"], "nearbyPlaces":["beach"], "maxPrice":5000 }
99  User Query: "${prompt}"
102 `;
```

- This is **prompt engineering**: a role ("query parser"), a strict output format, the allowed schema, type rules, and one example (**one-shot prompting**). The user's text is inserted with a template literal at L100.
- Because the user's text sits inside the prompt, someone could write "ignore the rules and…" (**prompt injection**). The defences below (whitelisting fields, escaping regex, JSON parse) limit what that can achieve: the worst outcome is a wrong filter, not database access.

```
105 const result = await model.generateContent(aiPrompt);
114 console.log(JSON.stringify(result, null, 2));
119 const response = result.response.text();
129 const cleanResponse = response.replace(/```json/g, "").replace(/```/g,
"").trim();
142 let filters = {};
144 try { filters = JSON.parse(cleanResponse); }
149 catch (parseError) { return res.status(500).json({ success: false,
message: "Invalid AI response format" }); }
```

- **L105** one API call to Gemini (network, so `await`).
- **L119** `.text()` extracts the generated text.
- **L129–133** defensive cleanup: removes markdown code fences in case the model wraps its JSON in them (JSON mode makes this rare, but it is cheap insurance).
- **L142–165** parse the text into an object; if the model returned invalid JSON, return 500 instead of crashing.

```
168 const allowedFields =
["city","hotelType","amenities","nearbyPlaces","maxPrice","tags","rating"];
181 Object.keys(filters).forEach((key) => { if (!allowedFields.includes(key))
delete filters[key]; });
```

- **Whitelist:** removes any key the model invented (or that an injected prompt produced, such as `"$where"`). Never trust AI output — treat it like user input.

```
198 const query = {};
201 if (filters.city)      query.city      = { $regex:
escapeRegex(filters.city), $options: "i" };
214 if (filters.hotelType) query.hotelType = { $regex:
escapeRegex(filters.hotelType), $options: "i" };
229 if (filters.maxPrice)  query.price     = { $lte: Number(filters.maxPrice)
};
241 if (filters.amenities && Array.isArray(filters.amenities) &&
filters.amenities.length > 0)
253                       query.amenities = { $all: filters.amenities };
262 if (filters.nearbyPlaces && Array.isArray(...) && length > 0)
query.nearbyPlaces = { $in: filters.nearbyPlaces };
283 if (filters.tags && Array.isArray(...) && length > 0)         query.tags
= { $in: filters.tags };
304 if (filters.rating)    query.rating    = { $gte: Number(filters.rating)
};
```

- The query object is built **conditionally**: only filters that exist are added, so `{}` (no filters) returns all hotels.
- **`$regex`** + **`$options: "i"`** — case-insensitive partial match, so `"Goa"` matches `"goa"` and also `"north goa"`.
- **`$lte`** (less than or equal) — "under ₹5000".
- **`$gte`** (greater than or equal) — "4 star and above".
- **`$all`** — the hotel must have **every** amenity asked for ("pool and wifi" means both).
- **`$in`** — the hotel must match **at least one** of the values (near a beach *or* airport is fine).
- `Array.isArray` guards against the model returning a string where an array was expected.
- **Gap:** `$all` / `$in` are exact, case-sensitive matches. Stored values are lowercase, but if Gemini returns `"Pool"` or `"swimming pool"`, nothing matches. Lowercasing the AI arrays before querying would help.

```
322 const hotels = await hotelModel.find(query).lean().sort({ createdAt: -1
});
331 return res.status(200).json({ success: true, message: "...", filters,
totalHotels: hotels.length, hotels });
347 } catch (error) { return res.status(500).json({ success: false, message:
error.message || "Server Error" }); }
```

- **`.lean()`** returns plain JavaScript objects instead of full Mongoose documents (no getters, `save()`, change tracking), which is faster and lighter for read-only results.
- The response includes the extracted `filters`, useful for showing the user how their sentence was understood.

Worked example. Input: "luxury hotel in goa with pool under 5000 near beach" → Gemini returns `{"city":"goa","hotelType":"luxury","amenities":["pool"],"nearbyPlaces":["beach"],"maxPrice":5000}` → MongoDB query:

```
{ city: { $regex: "goa", $options: "i" },
  hotelType: { $regex: "luxury", $options: "i" },
  price: { $lte: 5000 },
  amenities: { $all: ["pool"] },
  nearbyPlaces: { $in: ["beach"] } }
```

5. Frontend line by line

The React app has one global store (Zustand) for the logged-in user, three small Axios modules for the API, one route table, and page components that keep their own form state with `useState`. Tailwind `className` strings are styling only; this section explains the logic and skips purely visual class names.

5.1 Build setup

`package.json` — `"type": "module"` (ES modules: `import` / `export`). Scripts: `dev` (Vite dev server, port 5173), `build` (production bundle in `dist/`), `lint`, `preview`.

`vite.config.js`

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'
export default defineConfig({ plugins: [react(), tailwindcss()] })
```

- `react()` enables JSX transform and Fast Refresh (instant reload without losing state). `tailwindcss()` is the Tailwind v4 plugin: it scans files for class names and generates only the CSS used.

`index.html` — the single HTML page of this **SPA (single-page application)**. `<div id="root">` is where React renders; `<script type="module" src="/src/main.jsx">` loads the app. `index.css` is empty in the repo (Tailwind v4 normally needs `@import "tailwindcss";` in it, so check your deployed copy).

`src/main.jsx`

```
1 import { StrictMode } from 'react'
2 import { createRoot } from 'react-dom/client'
3 import App from './App.jsx'
4 import './index.css'
6 createRoot(document.getElementById('root')).render(<App />)
```

- `createRoot` is the React 18 API that mounts the app into `#root`. `StrictMode` is imported but not used (it would double-run effects in development to catch bugs).

5.2 `src/App.jsx`

```
1  import { BrowserRouter } from "react-router-dom";
2  import AppRoutes from "./routes/AppRoutes";
3  import { useEffect } from "react";
4  import { useAuthStore } from "./AuthStore";
6  function App() {
7    const fetchUser = useAuthStore((state) => state.fetchUser);
9    useEffect(() => {
10     fetchUser(); // auto-login using cookies
11   }, []);
13   return ( <BrowserRouter> <AppRoutes /> </BrowserRouter> );
18 }
```

- **L7** reads the `fetchUser` action from the store with a **selector**; the component re-renders only when that slice changes.
- **L9–11** `useEffect` with an empty dependency array `[]` runs once after the first render. It calls `/auth/me` so a returning user (cookie still present) is logged back in.
- **L13** `BrowserRouter` uses the HTML5 History API so URLs look like `/owner` (no `#`). On Netlify this needs a `_redirects` rule (`/* /index.html 200`) or refreshing a deep URL gives 404.

5.3 `src/AuthStore.js` — Zustand store

```
1  import { create } from "zustand";
2  import { loginUser, registerUser, getMe, logoutUser } from
"./api/auth.js";
4  export const useAuthStore = create((set) => ({
5    user: null,
6    loading: false,
8    login: async (data) => {
10     set({ loading: true });
12     const res = await loginUser(data);
14     const user = res?.data?.user;
16     if (!user) throw new Error("Invalid response from server");
20     set({ user, loading: false });
24     return res.data.user
25   } catch (err) { console.log(...); set({ loading: false }); return false
}
33   register: async (data) => { set({ loading: true }); const res = await
registerUser(data);
41     set({ user: res.data.user, loading: false }); return res.data.user;
49   } catch (err) { set({ loading: false }); return null; }
59   fetchUser: async () => { try { const res = await getMe(); const user =
res.data.user;
65     set({ user }); return user; } catch (error) { set({ user: null });
return null; } },
75   logout: async () => { await logoutUser(); set({ user: null }); },
79 }));
```

- **Zustand** is a small state library: `create` takes a function that receives `set` and returns the initial state plus actions. Any component calls `useAuthStore(selector)` — no Provider wrapper needed (unlike Context or Redux).
- **State:** `user` (null = logged out) and `loading`.
- **login** — calls the API, takes `res.data.user`, saves it, and returns it so the Login page can redirect by role. `?.` prevents a crash if the response is unexpected. On error it returns `false` instead of throwing.
- **register** — same pattern; returns `null` on failure.
- **fetchUser** — restores the session on refresh. Because the backend `/me` does not send a `user` key (section 4.1), `user` becomes `undefined` here.
- **logout** — calls the backend to blacklist the token and clear the cookie, then clears local state.

- `set({...})` **merges** into the state (shallow), like `setState` in class components.

5.4 API layer — `src/api/*.js`

```
// auth.js
const API = axios.create({ baseURL: 'https://ai-powered-hotel-
booking.onrender.com/api', withCredentials: true })
export const loginUser    = async (data) => API.post('/auth/login', data)
export const registerUser = async (data) => API.post('/auth/register', data)
export const getMe        = async () => API.get('/auth/me')
export const logoutUser   = async () => API.post('/auth/logout')

// hotel.js  (baseURL .../api/hotel)
export const createHotel = (data) => API.post("/create", data)      // data
is FormData
export const getHotels   = () => API.get("/getHotel")
export const bookHotel   = (hotelId, payload) => API.post(`/book/${hotelId}`,
payload).then(r => r.data)
export const listing     = () => API.get('/listing')
export const aiSearch    = (prompt) => API.post("/ai-search", { prompt })

// owner.js  (baseURL .../api)
export const fetchOwnerDashboard = () => API.get('/owner')
```

- `axios.create` makes a pre-configured instance so each call only gives the path.
- **`withCredentials: true`** — tells the browser to send cookies on cross-origin requests and accept `Set-Cookie` from the response. It must pair with `credentials: true` + an exact origin on the server.
- When the body is `FormData`, Axios lets the browser set `Content-Type: multipart/form-data; boundary=…` automatically, which multer needs.
- Axios **rejects** (throws) for any status outside 2xx, which is why pages read `error.response.data.message` in `catch`.
- The base URL is hard-coded three times; a better approach is one instance using `import.meta.env.VITE_API_URL`.
- `bookHotel` returns `res.data` while the others return the whole response — inconsistent but works.

5.5 `src/routes/AppRoutes.jsx`

```
14 const PrivateRoute = ({ children }) => {
15   const user = useAuthStore((state) => state.user);
16   return user ? children : <Navigate to="/login" />;
17 };
21 <Routes>
22   <Route path="/login"    element={<Login />} />
23   <Route path="/register" element={<Register />} />
25   <Route path="/"         element={<PrivateRoute><User /></PrivateRoute>}
/>
36   <Route path="/create"   element={<PrivateRoute><CreateHotel />
</PrivateRoute>} />
45   <Route path="/hotels"   element={<HotelList />} />
52   <Route path="/listing"  element={<Listing />} />
59   <Route path="/owner"    element={<PrivateRoute><Owner /></PrivateRoute>}
/>
68   <Route path="/user"     element={<User />} />
74   <Route path="/admin"    element={<PrivateRoute><Admin /></PrivateRoute>}
/>
83   <Route path="/book/:id" element={<BookHotel />} />
```

- **PrivateRoute** is a wrapper component (a **route guard**): if a user is in the store it renders the child page; otherwise `<Navigate>` redirects to `/login`. `children` is whatever JSX is nested inside it.
- It only checks "logged in", not role, so a guest can open `/owner` (the backend still limits the data).
- `/hotels`, `/listing`, `/user` and `/book/:id` are unguarded; the server's `authMiddleware` still protects the data calls.
- **Timing issue:** on refresh `user` starts `null` and `fetchUser` is asynchronous, so the guard redirects before the check finishes. A fix is an `authChecked` / `loading` flag that shows a spinner until `fetchUser` resolves.
- `:id` in `/book/:id` is a URL parameter read with `useParams()`.
- `HotelList` at `/hotels` is rendered without its `hotels` prop, so that route always shows the empty message.

5.6 Components — `src/componenet/`

`Navbar.jsx`

- Reads `logout` and `user` from the store with two selectors.
- `useNavigate()` returns a function for programmatic navigation.
- `handleLogout` calls `logout()` (not awaited) then `navigate("/login")`.
- Logo `onClick={() => navigate("/")}` returns home.
- **Conditional rendering:** `{user?.role === "owner" && <Link to="/create">Create Hotel</Link>}` — shown only to owners. `{user ? <button>Logout</button> : <Link>Login</Link>}` is a ternary switch. The role badge shows `user.role`.
- `<Link>` changes the URL without a full page reload (client-side routing).

`HotelCard.jsx` — receives one `hotel` via props (`function HotelCard({ hotel })` is prop destructuring).

- `src={hotel.img || "https://via.placeholder.com/400"}` — fallback image.
- `₹{hotel.price}` — JSX expression in braces.
- `{hotel.featured && <div>Featured</div>}` — badge only if true.
- `{hotel.rating || "4.5"}` — shows a fake 4.5 when the rating is 0 (0 is falsy), which can mislead users.
- `{hotel.hotelType && …}` and `{hotel.amenities?.length > 0 && …}` — render only if data exists.
- `hotel.amenities.slice(0, 3).map((item, index) => <div key={index}>…)` — first 3 amenities; every list item needs a unique `key` so React can track it between renders (using the value `item` would be better than the index).
- `<Link to={` /book/${hotel._id} `}>Book Now</Link>` — passes the hotel id in the URL.

`FormInput.jsx` — `function FormInput({ label, ...props })` renders a label and `<input {...props} />`. The rest/spread pattern forwards every other prop (type, value, onChange). It is not used by any page.

5.7 Pages

`Login.jsx`

```
7  const login = useAuthStore((s) => s.login);
9  const [form, setForm] = useState({ email: "", password: "" });
14 const handleSubmit = async (e) => {
15   e.preventDefault();
18   const user = await login(form);
20   if (user.role === "user") navigate("/user");
22   else if (user.role === "owner") navigate("/owner");
24   else if (user.role === "admin") navigate("/admin");
```

- **Controlled inputs:** `value={form.email}` + `onChange={e => setForm({ ...form, email: e.target.value })}`. React state is the single source of truth; the spread `...form` copies the other fields so they are not lost (state must be replaced immutably, not mutated).
- `e.preventDefault()` stops the browser's default full-page form submit.
- **Role-based redirect** after login. If login failed, `login` returns `false`, `false.role` is `undefined`, so nothing happens and no error is shown to the user.

`Register.jsx`

- Form state `{ username, email, password, role: "user" }`; a `<select>` offers `user` / `owner` (the backend whitelist ignores anything else).
- On success: `owner` → `/owner`, else `/user`. Uses `==` (loose equality); `===` is preferred.

`CreateHotel.jsx` (owner adds a hotel)

```
9   const [data, setData] = useState({ title:"", description:"", city:"",
address:"", price:"", room:"",
20    rating:"", maxGuests:"", hotelType:"", amenities:"", tags:"",
nearbyPlaces:"", policies:"",
30    checkInTime:"", checkOutTime:"", featured:false, img:null });
39  const handleChange = (e) => {
41    const { name, value, files, type, checked } = e.target;
44    if (name === "img") setData({ ...data, img: files[0] });
54    else if (type === "checkbox") setData({ ...data, [name]: checked });
64    else setData({ ...data, [name]: value });
75  const handleSubmit = async (e) => {
77    e.preventDefault();
81    const formData = new FormData();
84    Object.keys(data).forEach((key) => { formData.append(key, data[key]);
});
88    await createHotel(formData);
90    alert("Hotel created successfully!");
92    setData({ ...reset... });
121   navigate("/owner");
123 } catch (error) { alert("Something went wrong"); }
```

- **One generic `handleChange`** for every input, using the input's `name` attribute. `[name]: value` is a **computed property name**, so `name="city"` updates `data.city`.
- File inputs expose `e.target.files` (a FileList); `files[0]` is the chosen File. Checkboxes use `checked`, not `value`.
- **Why FormData?** JSON cannot carry binary files. `FormData` builds a `multipart/form-data` body where each field is a part; the image part is read by `upload.single('img')` on the server. Booleans and numbers become strings in FormData, which is why the backend converts them.
- Amenities, tags, nearby places and policies are typed as comma-separated text; the server splits them into arrays.
- `hotelType` `<select>` options match the schema enum exactly (budget, luxury, business, resort, hostel, villa); `checkInTime` / `checkOutTime` use `<input type="time">`.

`BookHotel.jsx`

```
15  const { id } = useParams();
19  const [form, setForm] = useState({ checkIn: "", checkOut: "",
selectedRooms: 1, guests: 1, specialRequests: "" });
33  const [loading, setLoading] = useState(false);
39  const roomPrice = 2000;
42  const totalNights = form.checkIn && form.checkOut ? Math.ceil((new
Date(form.checkOut) - new Date(form.checkIn)) / (1000*60*60*24)) : 0;
55  const totalPrice = totalNights > 0 ? roomPrice *
Number(form.selectedRooms) * totalNights : 0;
63  const handleBook = async () => {
66    if (!form.checkIn || !form.checkOut) { alert(...); return; }
77    if (new Date(form.checkOut) <= new Date(form.checkIn)) { alert(...);
return; }
88    const payload = { checkIn, checkOut, selectedRooms: Number(...),
guests: Number(...), specialRequests };
107   setLoading(true);
110   const response = await bookHotel(id, payload);
114   alert("Booking Confirmed ✅");  117 setForm(reset);  131 navigate("/");
137 } catch (error) { alert(error?.response?.data?.message || "Booking failed
❌"); }
142 finally { setLoading(false); }
```

- `useParams()` reads `:id` from the URL.
- `totalNights` and `totalPrice` are **derived values**, recalculated on every render from state, not stored in state.
- `roomPrice = 2000` is hard-coded (comment says it should come from a hotel-details API), so the price shown can differ from what the server charges. The server's price is the real one.
- Client validation gives fast feedback; the same checks exist on the server because client checks can be bypassed.
- `min={new Date().toISOString().split("T")[0]}` stops picking a past check-in date (in UTC; IST is 5:30 ahead, so near midnight it can be off by a day).
- `try / catch / finally` — `finally` always resets `loading`; the button is `disabled={loading}` to prevent double booking clicks.
- `error?.response?.data?.message` shows the server's message (e.g., "Requested rooms not available").

`HotelList.jsx` — receives `hotels` as a prop; if empty shows a message, otherwise a responsive CSS grid (`grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4`) of `HotelCard`s with `key={h._id}`. A **presentational component**: no state, no API calls.

`Listing.jsx` (owner's own hotels)

- `useEffect(..., [])` defines an inner async function and calls it (an effect callback itself cannot be `async` because it must return nothing or a cleanup function).
- `response?.data?.hotels || response?.data || []` then `Array.isArray(...)` — defensive parsing.
- `finally { setLoading(false) }`; shows loading, empty, or a grid. It displays `hotel.username`, which does not exist on a hotel, so that line is always blank (should be `hotel.title`).

`Dashboard/User.jsx` (guest home + AI search)

```
15 const [searchText, setSearchText] = useState("");
18 const [allHotels, setAllHotels] = useState([]);
21 const [aiHotels, setAiHotels] = useState([]);
24 const [loading, setLoading] = useState(false);
27 const [searched, setSearched] = useState(false);
30 useEffect(() => { getHotels().then((res) =>
setAllHotels(res.data.hotels)).catch(console.error); }, []);
47 const handleAISearch = async () => {
49   if (!searchText.trim()) { alert("Please enter search text"); return; }
58   setLoading(true);
60   const response = await aiSearch(searchText);
64   setAiHotels(response.data.hotels || []);
66   setSearched(true);
72 } catch { alert("AI Search Failed"); } finally { setLoading(false); }
81 const clearSearch = () => { setAiHotels([]); setSearchText("");
setSearched(false); };
```

- Two lists are kept: all hotels (loaded once on mount) and AI results. `searched` decides which one to show: `<HotelList hotels={searched ? aiHotels : allHotels} />`.
- `searchText.trim()` blocks empty or space-only searches.
- Suggestion chips ("Luxury hotel in Goa", "Budget hotel in Mumbai" …) call `setSearchText(item)` to prefill the box.
- While `loading`, the button shows a spinning `Loader2` icon.

- Header text uses nested ternaries: "AI Search Results" vs "Popular Hotels", and "Found N matching hotels" / "No matching hotels found" / "Showing N hotels".

`Dashboard/Owner.jsx`

- State: `stats` (object), `hotels`, `bookings`, `loading` (starts `true`).
- `useEffect(() => { fetchDashboard() }, [])` → `fetchOwnerDashboard()` → `GET /api/owner` → sets the three pieces of state.
- Shows four stat cards (`stats.totalHotels`, `totalRooms`, `totalBookings`, `₹{stats.totalRevenue}`), a properties list, and a bookings list with `booking.userId?.username` and `booking.hotelId?.title` (available because of `populate`). Status chip is green when `bookingStatus === "confirmed"`.
- Buttons: "+ Add Hotel" → `/create`; second button → `/listing`.
- `axios` is imported but unused.

`Dashboard/Admin.jsx` — a static UI. `stats`, `hotels` and `users` are hard-coded arrays (e.g., Users 1240, "Royal Palace", "Rahul Sharma"); there is no admin API yet. Say this honestly if asked: "the admin panel is a UI prototype; next step is admin endpoints for users and hotel approval".

6. API reference

The backend exposes 11 endpoints; all bodies are JSON except hotel creation, which is `multipart/form-data`.

Method	Full path	Auth	Body / params	Success response	Error codes
POST	`/api/auth/register`	none	`{ username, email, password, role? }`	201 `{ success, message, user }` + `token` cookie	400 missing, 409 exists, 500
POST	`/api/auth/login`	none	`{ email, password }`	200 `{ user: { _id, name, email, role } }` + cookie	400, 404 no user, 401 wrong password, 500
GET	`/api/auth/me`	cookie	—	200 `{ message, username, email }`	401

Method	Full path	Auth	Body / params	Success response	Error codes
POST	`/api/auth/logout`	cookie	—	200 `{ message }`, cookie cleared	401
POST	`/api/hotel/create`	cookie	multipart: hotel fields + file `img`	201 `{ message, hotel }`	400 missing / no image, 500 upload or DB
GET	`/api/hotel/getHotel`	none	—	200 `{ success, totalHotels, hotels }`	500
POST	`/api/hotel/book/:hotelId`	cookie	`{ checkIn, checkOut, selectedRooms, guests?, specialRequests? }`	201 `{ success, message, booking }`	400 invalid, 404 hotel, 500
DELETE	`/api/hotel/book/:hotelId`	cookie + admin	—	200 `{ success, message }`	401, 403, 404, 500 (currently always 500, see 7)
GET	`/api/hotel/listing`	cookie	—	200 `{ success, hotels }` (only mine)	401, 500
POST	`/api/hotel/ai-search`	none	`{ prompt }` (max 500 chars)	200 `{ success, filters, totalHotels, hotels }`	400 empty/too long, 500 AI or parse
GET	`/api/owner`	cookie	—	200 `{ success, stats, hotels, bookings }`	401, 500

Status codes used: **200** OK, **201** Created, **400** Bad Request (bad input), **401** Unauthorized (not logged in / bad token), **403** Forbidden (logged in, wrong role), **404** Not Found, **409** Conflict (duplicate), **500** Internal Server Error.

Environment variables the backend needs in `Backend/.env` (git-ignored): `MONGO_URI`, `JWT_SECRET`, `IMAGEKIT_PRIVATE_KEY`, `GOOGLE_GENAI_API_KEY`.

7. Bugs and weaknesses (know these before the viva)

The two that break visible features are the `/me` response shape (users are logged out on every refresh) and `authorize` reading `req.user` (admin delete always returns 500). Examiners often ask "what would you improve?" — this list is your answer.

#	Where	Problem	Fix
1	`getMeController`	Returns `{ username, email }`, frontend reads `res.data.user` → always null after refresh; no `role`; missing `return` when user not found	`return res.json({ user: { _id, username, email, role } })` and `return` on not-found
2	`authorize.js`	Reads `req.user.role` but middleware sets `req.role` → TypeError → 500	`roles.includes(req.role)`
3	`authMiddleware`	Never checks the token blacklist, so logout does not revoke the token	`if (await tokenBlackListModel.findOne({ token })) return 401`
4	Booking	Room count reduced permanently and ignores dates; never restored	Check overlapping bookings by date: `checkIn < newCheckOut && checkOut > newCheckIn`
5	Booking	Race condition (two users book the last room)	Atomic `findOneAndUpdate({ _id, room: { $gte: n } }, { $inc: { room: -n } })` + transaction
6	Booking	`guests` not checked against `maxGuests × rooms`; past dates accepted on server	Add server checks
7	`/hotel/create`, `/owner`	No owner role check	Add `authorize("owner")`

#	Where	Problem	Fix
8	`BookHotel.jsx`	Price hard-coded ₹2000	Fetch hotel by id (`GET /hotel/:id`)
9	`PrivateRoute`	Redirects before `fetchUser` finishes; no role check	`authChecked` flag + allowed-roles prop
10	`db.js` , `gemini.js`	Log `MONGO_URI` and API key	Remove the `console.log` s
11	`loginController`	Different messages reveal which emails exist; cookie has no `maxAge`	Generic "Invalid credentials"; add `maxAge`
12	`logoutController`	Second logout with same token throws E11000 (no try/catch); `clearCookie` without matching options	try/catch or `updateOne(..., { upsert: true })` ; pass `secure` , `sameSite`
13	multer	No file size or type limit	`limits: { fileSize: 2*1024*1024 }` , `fileFilter` for images
14	`/ai-search`	Public and unlimited → Gemini cost abuse	`express-rate-limit` ; lowercase AI arrays before `$all` / `$in`
15	Delete hotel	Orphan bookings; misleading path `/book/:hotelId`	`DELETE /hotel/:id` , delete or cancel related bookings
16	Inputs	No email format / password length validation; `findById` CastError gives 500	`express-validator` or Joi/Zod; `mongoose.isValidObjectId`
17	`server.js`	Port hard-coded; server starts before DB connects	`process.env.PORT` ; `await connectDB()` then `listen`

#	Where	Problem	Fix
18	Frontend	Base URL hard-coded 3 times; `Listing` shows `hotel.username`; `HotelCard` shows fake 4.5 rating; `/hotels` route gets no data	`VITE_API_URL`; `hotel.title`; show "New"; pass data
19	Admin	Dashboard is dummy data	Admin APIs: list users, block, approve hotels
20	No payments, no cancel, no "my bookings" for users, no tests	Add `GET /bookings/me`, `PATCH /bookings/:id/cancel`, Razorpay/Stripe, Jest + Supertest	

8. Expected viva / interview questions with answers

Answer in your own words; each answer below is the core point an examiner listens for.

8.1 Project and architecture

1. **Explain your project in 30 seconds.** StayAI is a MERN hotel booking app with user, owner and admin roles. Owners list hotels with images on ImageKit, users book them, and an AI search uses Google Gemini to convert a plain-English query into MongoDB filters. Auth is JWT in an httpOnly cookie.
2. **What does MERN mean?** MongoDB (database), Express (backend framework), React (frontend), Node.js (JavaScript runtime on the server).
3. **Why MongoDB over SQL?** Hotel data has flexible, list-like fields (amenities, tags, nearby places) that map naturally to arrays in a document, and JSON documents match JavaScript objects end to end. For strongly transactional booking at scale, SQL would also be a valid choice.
4. **What architecture does the backend follow?** Layered MVC style: routes → middleware → controllers → models, plus services for external APIs and config for connections.
5. **Why separate `app.js` and `server.js`?** `app.js` builds the Express app; `server.js` starts it. The app can then be imported in tests without opening a port.

6. **Walk through what happens when a user clicks Book Now.** See section 1: React → Axios with cookie → CORS/JSON/cookie middleware → hotel router → `authMiddleware` → `bookingHotelController` → MongoDB → 201 JSON → alert and redirect.
7. **Where is it deployed?** Frontend on Netlify, backend on Render, database on MongoDB (Atlas), images on ImageKit.

8.2 Schemas and MongoDB

8. **How many collections and how are they related?** Four: users, hotels, books, tokenblacklists. Hotel → user via `createdBy`; booking → user via `userId` and → hotel via `hotelId`.
9. **Embedding vs referencing — which did you use and why?** Referencing. Bookings grow without limit and are queried on their own (owner dashboard), so embedding them in hotels or users would make documents huge (16 MB limit) and duplicate data.
10. **What is `ref` and what does `populate()` do?** `ref` names the model an ObjectId points to. `populate("userId", "username email")` replaces the id with that user's selected fields by running an extra query.
11. **What does `timestamps: true` do?** Adds and auto-updates `createdAt` and `updatedAt`.
12. **What is an index? Which ones exist?** A sorted lookup structure (B-tree) that avoids scanning every document. Indexes: unique `email`; unique `title`, `city`, `price` on hotels; `userId`, `hotelId` on bookings; unique `token` and TTL `createdAt` on the blacklist; plus the default `_id`.
13. **Cost of indexes?** Extra storage and slower writes, since every insert/update must update each index.
14. **Is `unique` a validator?** No. It creates a unique index; duplicates fail with MongoDB error E11000, which is why the controller checks `findOne` first to send a friendly 409.
15. **What is a TTL index?** An index with `expireAfterSeconds`; MongoDB's background task deletes documents once that time passes after the indexed date. Used to auto-clean blacklisted tokens after 3 days.
16. **Why `pricePerNight` in the booking?** A price snapshot, so a later price change does not alter past bookings.
17. **Why `lowercase: true` on arrays?** MongoDB string matching is case-sensitive; storing lowercase makes "WiFi" and "wifi" match.

18. **What does `enum` do?** Rejects values not in the list at validation time (hotelType, role, bookingStatus).
19. **Difference between `find`, `findOne`, `findById`?** `find` returns an array of all matches; `findOne` the first match or null; `findById(id)` is `findOne({ _id: id })`.
20. **What is `.lean()`?** Returns plain JS objects instead of Mongoose documents — faster, less memory, but no `save()` or virtuals. Good for read-only lists.
21. **Explain `$in`, `$all`, `$lte`, `$gte`, `$regex`.** `$in`: field matches any value in the list. `$all`: array field contains every listed value. `$lte` / `$gte`: less/greater than or equal. `$regex` with `$options: "i"`: case-insensitive pattern match.
22. **What is `__v`?** Mongoose's version key, incremented on certain array updates to detect conflicting saves.
23. **How would you get total revenue in one query?** Aggregation: `$match` owner's hotels → `$lookup` bookings → `$group` with `$sum: "$totalPrice"`.

8.3 Authentication and security

24. **How does login work end to end?** Find user by email → `bcrypt.compare` the password → `jwt.sign({ userId, role })` with a secret, 3-day expiry → set as httpOnly cookie → return user and role → frontend stores the user in Zustand and redirects by role.
25. **What is a JWT? Its three parts?** JSON Web Token: `header.payload.signature`, base64url-encoded. Header holds the algorithm, payload holds claims (`userId`, `role`, `iat`, `exp`), signature is an HMAC of the first two parts using the secret.
26. **Is the JWT payload encrypted?** No, only signed. Anyone can decode it, so never put passwords in it. The signature only proves it was not changed.
27. **Why store the token in an httpOnly cookie instead of localStorage?** JavaScript cannot read httpOnly cookies, so an XSS attack cannot steal the token. localStorage is readable by any script on the page.
28. **Then what about CSRF?** Cookies are sent automatically, so CSRF is the risk with cookie auth. Mitigations: `sameSite` settings, CORS whitelist with credentials, CSRF tokens. Here `sameSite: "none"` is needed because frontend and backend are on different domains, so a CSRF token or same-domain deployment would strengthen it.
29. **Explain `secure`, `sameSite`, `maxAge`.** `secure`: HTTPS only. `sameSite: "none"`: send on cross-site requests (requires secure). `maxAge`: lifetime in ms; without it the cookie dies when the browser closes.

30. **Why hash passwords? Why bcrypt?** If the DB leaks, hashes cannot be reversed. bcrypt adds a random salt per password (same password → different hash) and is deliberately slow; cost factor 10 means 2^10 rounds.
31. **What is a salt?** Random data added before hashing so identical passwords give different hashes and precomputed rainbow tables are useless. bcrypt stores it inside the hash string.
32. **How does `bcrypt.compare` work if hashing is one-way?** It extracts the salt from the stored hash, hashes the entered password with it, and compares the results.
33. **What is middleware? What does `next()` do?** A function run before the controller that can read/modify `req`, end the response, or call `next()` to continue the chain.
34. **401 vs 403?** 401: not authenticated (no or bad token). 403: authenticated but not allowed (wrong role).
35. **How is role-based access done?** `authorize(...roles)` is a middleware factory returning a middleware that checks the user's role against the list. (Mention the `req.user` vs `req.role` bug and fix.)
36. **Can a user register as admin?** No. The controller whitelists `user` and `owner`; any other role becomes `user`.
37. **Why a token blacklist? Why is JWT logout hard?** JWTs are stateless and valid until expiry; deleting the cookie does not invalidate a copied token. A blacklist stores logged-out tokens so the server can reject them. (Admit the middleware does not yet check it.)
38. **Alternatives to a blacklist?** Short-lived access tokens (15 min) + refresh tokens stored in DB; or server-side sessions.
39. **What is CORS and why `credentials: true`?** Browsers block cross-origin responses unless the server allows the origin. `credentials: true` allows cookies; it requires an exact origin list, not `*`.
40. **What is prompt injection and how do you limit it?** A user writing instructions into the search box to make the AI misbehave. Defences: JSON mode, a whitelist of allowed fields, type checks, regex escaping, and never giving the AI database access — it only produces filters.
41. **What is NoSQL injection?** Sending objects like `{ "$ne": null }` where a string is expected. Field whitelisting, `Number()` casting and Mongoose schema casting reduce it; `express-mongo-sanitize` adds more.
42. **Why `escapeRegex`?** So special characters in a city name are matched literally and cannot form slow or broad regex patterns (ReDoS).

43. **Where are secrets kept?** In `.env`, loaded by dotenv and git-ignored. (Admit the console.logs of secrets should be removed.)

8.4 AI search

44. **How does AI search work?** User sentence → prompt to Gemini (`gemini-2.5-flash-lite`, JSON mode) → JSON filters → clean and parse → keep allowed fields → build a MongoDB query with `$regex`, `$lte`, `$all`, `$in`, `$gte` → `find().lean().sort()` → return hotels and filters.
45. **Why not let the AI query the database directly or pick hotels?** It could hallucinate hotels, is slower and costlier with all data in the prompt, and is unsafe. Using AI only to parse keeps results accurate and the database secure.
46. **What is `responseMimeType: "application/json"`?** Gemini's JSON mode: forces the reply to be JSON, making parsing reliable.
47. **Why still strip ``` fences?** Defensive programming; models sometimes wrap JSON in markdown.
48. **What if Gemini returns invalid JSON?** `JSON.parse` throws; the inner try/catch returns 500 "Invalid AI response format".
49. **What is one-shot prompting?** Giving one example output in the prompt to show the exact format.
50. **Why `flash-lite`?** Fast, low cost, enough for simple extraction.
51. **Why `$all` for amenities but `$in` for nearby places?** Amenities are requirements (pool AND wifi); nearby places and tags are preferences where any match is useful.
52. **Limitations?** Exact-match arrays miss synonyms ("swimming pool" vs "pool"), latency of an external API call, cost, and no rate limit. Improvements: lowercase/synonym mapping, text index or vector (semantic) search with embeddings, caching common queries.
53. **What if the Gemini API is down?** The catch returns 500. A fallback could run a simple keyword/`$text` search.

8.5 Booking logic

54. **How are nights calculated?** `(checkOut − checkIn)` in ms ÷ 86,400,000, rounded up with `Math.ceil`.
55. **Why compute price on the server?** Client values can be tampered with in DevTools or Postman; the server is the source of truth.

56. **How do you prevent overbooking?** Currently by comparing requested rooms with `hotel.room` and decrementing. Weaknesses: not date-aware and a race condition. Proper fix: count overlapping bookings for the date range and use an atomic update or transaction.
57. **What is a race condition here?** Two requests read the same room count before either saves, so both succeed and rooms go negative.
58. **What is a MongoDB transaction?** A group of operations that all succeed or all roll back (ACID), via `session.startTransaction()`; needs a replica set (Atlas has one).
59. **How would you add cancellation?** `PATCH /bookings/:id/cancel`: check the booking belongs to the user, set `bookingStatus: "cancelled"`, add rooms back with `$inc`, and exclude cancelled from revenue.
60. **How would you add payments?** Create a Razorpay/Stripe order on the server, create the booking as `pending`, verify the payment signature in a webhook, then mark it `confirmed`.

8.6 File upload

61. **How is the image uploaded?** React `FormData` → multipart request → `multer.memoryStorage()` puts the file in `req.file.buffer` → convert to base64 → ImageKit SDK upload → store returned URL in `hotel.img`.
62. **Why not store images in MongoDB?** Large binaries bloat the DB and slow queries; a CDN serves images faster and can resize them.
63. **memoryStorage vs diskStorage?** Memory keeps the file in RAM (simple, good for forwarding to a cloud), disk writes to the server (needed for very large files; Render's disk is temporary anyway). Memory needs a size limit to avoid running out of RAM.
64. **What does `upload.single('img')` mean?** Accept one file from the form field named `img`; other fields go to `req.body`.
65. **Why does the controller use `Number(price)`?** Multipart fields always arrive as strings.

8.7 React and frontend

66. **Why Zustand instead of Redux or Context?** Very little boilerplate, no Provider, components re-render only for the slice they select. Redux is heavier; Context re-renders every consumer on any change.
67. **What is a selector in `useAuthStore((s) => s.user)`?** A function picking part of the state; the component re-renders only when that part changes.

68. **What does `useEffect(fn, [])` do?** Runs once after the first render — used to fetch data and restore login.
69. **Why can't the `useEffect` callback be async?** It must return nothing or a cleanup function, but async returns a Promise. So an inner async function is defined and called.
70. **What is a controlled component?** An input whose value comes from React state and updates via `onChange`.
71. **Why `{ ...form, email: value }`?** State must be updated immutably; the spread copies the other fields so they aren't lost.
72. **What is `[name]: value`?** A computed property key, letting one `handleChange` serve all inputs.
73. **Why `e.preventDefault()`?** Stops the browser's full page reload on form submit.
74. **How does `PrivateRoute` work?** If a user exists in the store, render the children; otherwise `<Navigate to="/login" />`.
75. **Is frontend route protection enough?** No. Anyone can bypass the UI; real protection is the backend middleware. Frontend guards are only for user experience.
76. **Why do lists need a `key`?** So React's reconciliation can match items across renders; prefer a stable id like `_id` over the index.
77. **What is `withCredentials: true`?** Makes Axios send and accept cookies on cross-origin requests.
78. **Why `BrowserRouter`, and what breaks on refresh in production?** Clean URLs via the History API; the host must redirect all paths to `index.html` (Netlify `_redirects`).
79. **What is Vite?** A build tool and dev server using native ES modules for instant startup and Rollup for production builds.
80. **What is Tailwind?** A utility-first CSS framework; styles are composed with small classes in JSX.
81. **Props vs state?** Props are inputs passed from a parent (read-only); state is data a component owns and changes.
82. **What is conditional rendering?** Showing JSX based on a condition with `&&` or the ternary `? :` (e.g., "Create Hotel" only for owners).

8.8 Node and Express basics

83. **What is Express?** A minimal Node.js web framework for routing and middleware.
84. **`app.use` vs `router.get`?** `app.use` mounts middleware or routers for a path prefix and all methods; `router.get` handles one method on one path.

85. **`req.params` vs `req.query` vs `req.body`?** Params from the URL path (`/book/:hotelId`), query from `?key=value`, body from the request payload.
86. **What does `express.json()` do?** Parses JSON request bodies into `req.body`.
87. **async/await and why try/catch?** `await` pauses until a Promise settles; try/catch handles rejected Promises so errors return clean JSON instead of crashing.
88. **What is dotenv?** Loads `.env` key-value pairs into `process.env`.
89. **CommonJS vs ES modules?** Backend uses `require` / `module.exports` (CommonJS); frontend uses `import` / `export` (ESM).
90. **Why is Node good for this?** Non-blocking I/O suits an API that mostly waits on the database, ImageKit and Gemini.

8.9 "What would you improve?" — short answer

Fix `/me` and `authorize`; check the blacklist in middleware; date-based availability with atomic updates and transactions; owner role guards; real hotel price on the booking page; user "my bookings" and cancellation; payment gateway; input validation (Joi/Zod); rate limiting on AI search; pagination; admin APIs; tests with Jest + Supertest; env-based URLs; remove secret logs.